SIC PROJECT REPORT

On

AI Research Agent (RAG): LLM-powered tool for querying private document PDF sets

Submitted by:

Siddharth Sinha
Enroll No. - SIC202502113
Bachelor of Technology
In
Computer Science and Engineering

Under the guidance of

Dr Sweta Tripathi

ASET, Amity University, Haryana

Department of Computer Science &Engineering
Amity School of Engineering & Technology
Amity University Haryana, Gurugram, India

Department of Computer Science and Engineering

Amity School of Engineering and Technology

DECLARATION

I, **Siddharth Sinha**, student of B-Tech. Computer Science and Engineering hereby declare that the project entitled **"AI Research Agent (RAG): LLM-powered tool for querying private document PDF sets"** which is submitted by me to department of Computer Science & Engineering, Amity School of Engineering & Technology, Amity University Haryana, in partial fulfillment of the requirement for the award of the degree of Bachelors of Technology in Computer Science and Engineering, has not been previously formed the basis for the award of any degree, diploma or other similar title or recognition. I have taken care in all respects to honor the intellectual property rights and have acknowledged the contributions of others for using them. I further declare that in case of any violation of intellectual property rights or copyrights, I as a candidate will be fully responsible for the same. My supervisor, Head of department and the Institute should not be held for full or partial violation of copyrights if found at any stage of my degree.

Siddharth Sinha

A50105223148

SIC202502113

Department of Computer Science and Engineering

Amity School of Engineering and Technology

CERTIFICATE

This is to certify that the work in the project report entitled **"AI Research Agent (RAG): LLM-powered tool for querying private document PDF sets"** by Siddharth Sinha A50105223148 is a Bonafede record of project work carried out by him under our supervision and guidance in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering in the Department of Computer Science and Engineering, Amity School of Engineering and Technology, Amity University Haryana, Gurgaon. Neither this project nor any part of it has been submitted for any degree or academic award elsewhere.

Dr. Sweta Tripathi
Associate Professor
Computer Science & Engineering
ASET, Amity University, Haryana

Department of Computer Science and Engineering

Amity School of Engineering and Technology

ACKNOWLEDGEMENT

I would like to extend my heartfelt gratitude to my mentor, Dr. Sweta Tripathi, from the Amity School of Engineering & Technology, Amity University Haryana. Dr. Sweta Ma'am's unwavering support and encouragement have been instrumental in the successful completion of this project. It is his guidance that has elevated my work to its highest potential.

I am deeply thankful to my close-knit group of friends who provided me with constant inspiration, offered valuable guidance, and supervised the project's progress. Their contributions have been invaluable in shaping this project.

Special thanks go to my project supervisor, whose expert advice and consistent support throughout the project have been the cornerstone of its success.

Last but not least, I owe a debt of gratitude to my parents, whose unwavering support and inspiration have been a driving force behind my accomplishments. With utmost appreciation, I express my sincere thanks to them.

Siddharth Sinha
A50105223148
SIC202502113

ABSTRACT

The AI Research Agent Framework is a Python-based project developed to perform intelligent question answering over private PDF documents through an LLM-powered Retrieval-Augmented Generation (RAG) pipeline. By integrating classical NLP techniques, hybrid retrieval, and lightweight local language models through Ollama, the framework allows users to ingest documents, ask natural language questions, and receive structured, human-readable answers grounded in the source content. The system continuously processes user queries, extracts relevant context from indexed documents, selects the most useful chunks through dense and lexical retrieval, and generates answers with minimal manual effort.

This project leverages modern AI techniques, including custom neural embedding models, hybrid search, and large language models (LLMs) for context-based answer generation and interpretation. It preprocesses PDF text using tokenization, stopword removal, POS tagging, and lemmatization, stores chunked representations for efficient retrieval, and formats results in a clear and understandable manner. Additionally, the framework is modular in design, allowing future integration of improved retrievers, rerankers, interfaces, and additional local or API-based LLMs without major changes to the core architecture.

The AI Research Agent Framework has applications in private document search, technical manual assistance, educational content exploration, enterprise knowledge retrieval, and research support. Its ability to combine AI reasoning, semantic retrieval, and local LLM generation makes it a scalable, efficient, and intelligent solution for document understanding. By reducing the complexity of manually reading and searching large PDF collections, the framework improves productivity while maintaining grounding in source documents.

Python, along with supporting libraries such as NLTK, TensorFlow, Keras, scikit-learn, PyMuPDF, and Ollama, plays a crucial role in the development of the framework. These tools enable preprocessing, chunking, embedding generation, retrieval, and lightweight local answer generation, making the AI Research Agent Framework both a modern and practical solution for private document question answering.

Table of Contents

TITLE	PAGE NO.
Declaration	i
Certificate	ii
Acknowledgement	iii
Course Completion Certificate	iv
Abstract	v
Table of Contents	vi
Chapter - 1 Introduction	1
1.1 Aims & Objectives	2
Chapter - 2 Literature Review & Background Study	3
2.1 Literature Review	3
2.2 Background Study	4
2.2.1 Introduction to Retrieval-Augmented Generation (RAG)	4
2.2.2 Private Document Question Answering Systems	5
2.2.3 Role of NLP in Document Understanding	6
2.2.4 Role of LLMs in Intelligent Answer Generation	7
Chapter - 3 Technology Used	8-9
3.1 Python Programming Language	8
3.2 NLTK for Text Preprocessing	8
3.3 TensorFlow / Keras for Embedding Model	9
3.4 scikit-learn for TF-IDF and Retrieval	9
3.5 PyMuPDF for PDF Text Extraction	9
3.6 Ollama for Local LLM Integration	10
Chapter - 4 Design of Project	10-16
4.1 Hardware Requirements	12
4.2 Software Requirements	12
4.3 Methodology	13-15
4.3.1 PDF Extraction and Text Processing	13
4.3.2 Chunking and Vector Representation	14
4.3.3 Hybrid Retrieval and Answer Generation	15
4.4 Implementation	16-20
4.4.1 Data Ingestion and Preprocessing Pipeline	16-17
4.4.2 Embedding Model Training	17-18
4.4.3 Retrieval and Reranking System	18-19
4.4.4 Ollama Integration and Extractive Fallback	19-20
Chapter - 5 Results	21-23
5.1 System Interface and Setup	21
5.2 Query Processing and Answer Generation Results	22
5.3 Retrieval and Model Performance Observations	23
Chapter - 6 Conclusion and Future Scope	24-25
References	26

CHAPTER 1
INTRODUCTION

The AI Research Agent is an advanced document intelligence framework designed to perform question answering over private PDF document collections through an LLM-powered Retrieval-Augmented Generation (RAG) pipeline. By integrating classical Natural Language Processing (NLP), custom embedding models, hybrid retrieval techniques, and lightweight local language models, it provides an effective solution for extracting, understanding, and answering questions from unstructured document data. The framework is capable of processing natural language queries, retrieving the most relevant context from document collections, and generating structured, human-readable answers grounded in the source content.

Document question answering has become increasingly important in academic, organizational, and enterprise environments, where valuable information is often stored in manuals, reports, technical documents, and notes. Traditionally, finding information in such documents requires manual searching, which is time-consuming and inefficient. Although keyword-based search can help to some extent, it often fails to capture semantic meaning and context. The AI Research Agent simplifies this process by combining semantic retrieval and lexical retrieval with LLM-based answer generation, enabling users to interact with their documents more intelligently without requiring advanced technical expertise.

In modern AI systems, automation, contextual understanding, and grounded reasoning are essential. The AI Research Agent addresses these needs by maintaining a searchable knowledge base of processed PDF content, supporting trained embeddings for semantic similarity, and generating answers using lightweight local LLMs through Ollama. It also supports extractive fallback answering when generation is unavailable, making the system more robust and practical. By combining retrieval accuracy with LLM-based synthesis, the framework becomes suitable for students, researchers, technical users, and organizations that need efficient access to information from private documents.

The benefits of the AI Research Agent are as follows:

- **Automated Document Understanding:** Reduces manual effort by allowing users to ask natural language questions directly on private PDF collections.
- **Intelligent Retrieval:** Uses hybrid retrieval methods to identify the most relevant information from large document sets.

- **Grounded Answer Generation:** Generates answers based only on retrieved document context, reducing irrelevant or unsupported responses.
- **Efficiency & Productivity:** Saves time by combining ingestion, retrieval, and question answering into a single framework.
- **Extensibility:** Supports modular improvements such as better retrievers, rerankers, additional document types, and stronger local LLMs.
- **Private & Local Usage:** Supports lightweight local LLM integration through Ollama, making it useful for privacy-sensitive environments.

1.1 Aim and Objectives

The aim of the project **"AI Research Agent"** is to design and implement an LLM-powered Retrieval-Augmented Generation (RAG) system capable of answering user questions from private PDF document collections. The framework is intended to accept user queries in natural language, retrieve the most relevant document content using hybrid retrieval, and generate structured, grounded answers in a clear and understandable form. By integrating classical NLP, custom embeddings, hybrid search, and lightweight local LLM generation, the AI Research Agent reduces manual effort, improves efficiency, and assists users in extracting useful information from large document sets.

Objectives:

1. To gain a deeper understanding of Retrieval-Augmented Generation (RAG), document retrieval, and question-answering systems in the field of artificial intelligence.
2. To build a system that can process and analyze private PDF documents by extracting, preprocessing, and chunking textual content.
3. To develop an AI-powered framework that interprets natural language questions and retrieves the most relevant document context using hybrid retrieval methods.
4. To generate grounded answers using a lightweight local LLM and provide extractive fallback responses when generation is unavailable.
5. To maintain a modular and extensible architecture that supports future integration of better retrievers, rerankers, interfaces, and additional document sources.
6. To improve productivity by automating document search, retrieval, and answer generation, thereby reducing the need for manual reading and searching of large PDF collections.

CHAPTER 2

2.1 LITERATURE REVIEW

Document retrieval and question-answering systems have become an important area of research in artificial intelligence, especially with the rise of large language models and Retrieval-Augmented Generation (RAG). The core principle of such systems is to retrieve relevant context from a document collection and use that context to produce grounded, meaningful answers for user queries. Over time, document understanding has evolved from simple keyword-based search to semantic retrieval, neural embeddings, hybrid search, and LLM-based generation. This evolution has made information access faster, more accurate, and more useful in academic, enterprise, and research environments.

"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" – Lewis et al.

Abstract — Retrieval-Augmented Generation (RAG) combines parametric knowledge stored in language models with non-parametric knowledge retrieved from external sources. This approach improves factual grounding and reduces hallucination by enabling the model to generate answers based on retrieved evidence. The study demonstrates that combining retrieval with generation is highly effective for question answering and knowledge-intensive natural language tasks.

"Dense Passage Retrieval for Open-Domain Question Answering" – Karpukhin et al.

Abstract — Dense Passage Retrieval (DPR) introduced the idea of using neural embeddings to retrieve semantically relevant passages for open-domain question answering. Unlike traditional keyword-based search, dense retrieval captures deeper contextual meaning and improves passage relevance. The paper shows that embedding-based retrieval can significantly improve the performance of downstream question-answering systems.

"A Survey of Large Language Models" – Recent LLM Survey Papers

Abstract — Large Language Models (LLMs) have transformed NLP by enabling natural language understanding, reasoning, summarization, and answer generation at scale. However, these models often face limitations such as hallucination and lack of grounding. The survey highlights the importance of combining LLMs with retrieval systems so that generated answers remain connected to source evidence, especially in domain-specific and private document applications.

"Hybrid Search: Combining Lexical and Semantic Retrieval" – Information Retrieval Research

Abstract — Hybrid retrieval methods combine lexical search techniques such as TF-IDF or BM25 with semantic retrieval using neural embeddings. This combination improves performance because lexical methods are strong in exact term matching, while semantic methods capture contextual meaning. The literature shows that hybrid retrieval is particularly effective for technical and structured documents, where both terminology and context matter.

"Question Answering over Documents using NLP and Neural Networks" – Various Academic Studies

Abstract — Research in document question answering emphasizes the importance of preprocessing, chunking, indexing, retrieval, and answer generation. Studies show that tokenization, stopword removal, lemmatization, and chunk-based indexing improve the quality of retrieval. Neural networks and language models further improve system performance by enabling semantic understanding and natural language answer generation.

From the literature, it is clear that modern document QA systems are moving toward frameworks that combine classical NLP, embedding-based retrieval, hybrid search, and LLM-powered answer generation. This directly supports the design of the AI Research Agent, which uses preprocessing, chunking, a Keras-based embedding model, hybrid retrieval, and lightweight local LLM integration to answer questions from private PDF document collections.

2.2 BACKGROUND STUDY

2.2.1 INTRODUCTION TO RETRIEVAL-AUGMENTED GENERATION (RAG)

Retrieval-Augmented Generation, commonly referred to as **RAG**, is a framework in artificial intelligence that combines information retrieval with language generation. Instead of depending only on the internal knowledge of a language model, a RAG system first retrieves relevant information from an external knowledge source and then uses that retrieved context to generate an answer. This makes RAG highly useful for question answering, document analysis, enterprise search, and private knowledge retrieval.

In modern AI systems, RAG is important because large language models may sometimes produce incorrect or unsupported answers when they rely only on their pretrained knowledge. By grounding the answer in retrieved documents, RAG improves factual accuracy, reduces hallucination, and makes the output more explainable. A typical RAG pipeline includes document ingestion, preprocessing, chunking, indexing, retrieval, reranking, and final answer generation.

For private document question answering, RAG is especially valuable because it allows users to query their own PDFs, reports, manuals, and notes without exposing the data to a public system. By combining retrieval techniques with LLM-based generation, a RAG system can return answers that are both natural in language and supported by source context. Before diving into the design and implementation of the AI Research Agent, it is important to understand the foundational concepts of RAG, document retrieval, NLP preprocessing, and local LLM integration. This background study provides a structured overview of the technologies and approaches used in the project.

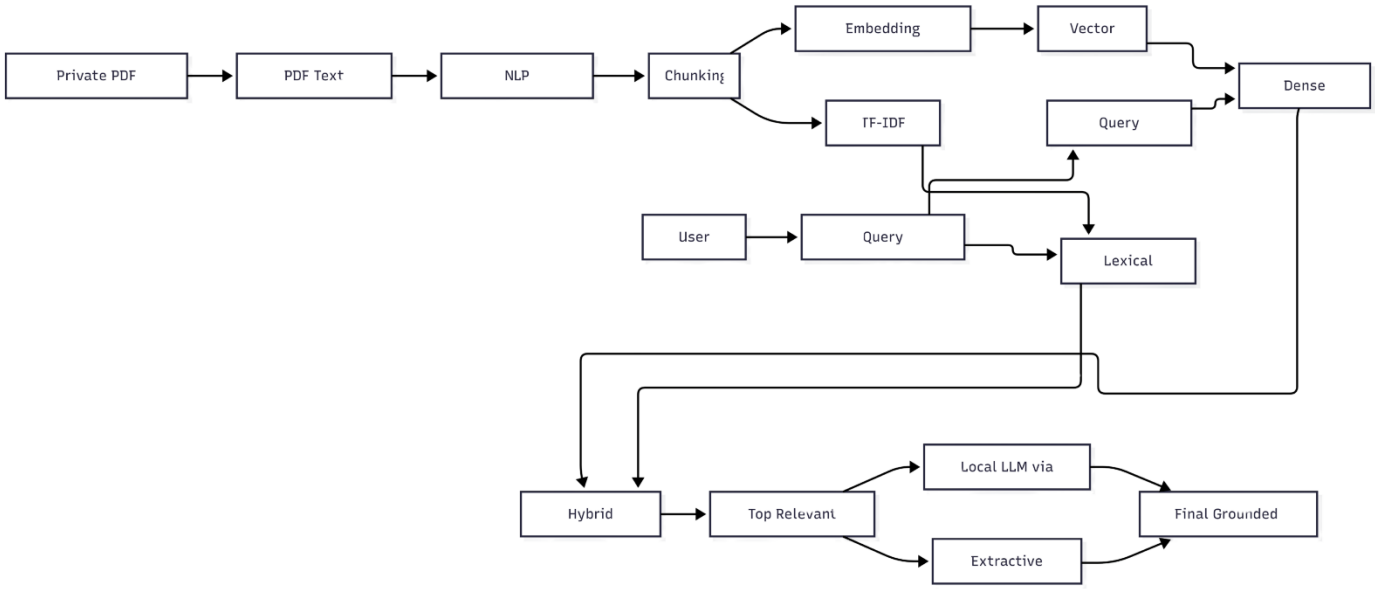


Fig. 1 Workflow

2.2.2 PRIVATE DOCUMENT QUESTION ANSWERING SYSTEMS

Private document question answering systems are designed to help users retrieve and understand inform ation from their own document collections, such as PDF reports, manuals, notes, research papers, and te chnical documents. Unlike open-domain question-answering systems that depend on public web scale data, private document QA systems operate on a specific, user controlled knowledge base. Their main objective is to provide accurate and context aware answers from the uploaded or ingested documents.

These systems are especially useful in academic, enterprise, and technical environments where large am ounts of important information are stored in unstructured documents. Manually reading and searching th rough such files is time-consuming and inefficient. Traditional search methods based only on keywords may miss important info rmation if the wording in the query differs from the wording in the document. Therefore, private docume nt QA systems often combine semantic retrieval, lexical retrieval, and answer-generation methods to improve relevance and usability.

A modern private document QA system usually includes document extraction, text preprocessing, chunk ing, indexing, retrieval, reranking, and answer generation. The AI Research Agent follows this same app roach by allowing users to query private PDF collections in natural language, retrieving the most relevan t document context, and generating grounded answers from that context.

2.2.3 ROLE OF NLP IN DOCUMENT UNDERSTANDING

Natural Language Processing (NLP) plays a central role in document understanding because it transforms raw text into a form that can be processed, indexed, and retrieved efficiently. Documents often contain noise such as punctuation, repeated words, formatting artifacts, and multiple grammatical forms of the same concept. NLP helps normalize this text so that the system can better understand and compare it.

In this project, NLP is used in several preprocessing steps, including tokenization, stopword removal, part-of-speech tagging, and lemmatization. **Tokenization** splits text into sentences and words. **Stopword removal** eliminates frequently occurring words that add little meaning, such as “the”, “is”, and “and”. **POS tagging** identifies the grammatical role of each word, while **lemmatization** converts words into their dictionary base form, such as “running” to “run” or “studies” to “study”.

These NLP techniques improve document understanding by reducing noise, normalizing vocabulary, and increasing the consistency of retrieval. They also help build better chunk representations for embedding models and lexical retrieval methods. As a result, NLP serves as the foundation for accurate indexing, semantic retrieval, and grounded question answering in the AI Research Agent framework.

2.2.4 ROLE OF LLMS IN INTELLIGENT ANSWER GENERATION

Large Language Models (LLMs) have transformed intelligent answer generation by enabling machines to produce fluent, context-aware, and human-like responses to user queries. In document question-answering systems, LLMs are especially valuable because they can synthesize retrieved information into concise and understandable answers instead of simply returning raw document excerpts.

However, LLMs also have limitations. If used without retrieval, they may rely too heavily on pretrained knowledge and generate answers that are not supported by the user's actual documents. This is why combining LLMs with retrieval is important. In a RAG framework, the LLM does not answer blindly. Instead, it receives relevant document context from the retrieval system and generates an answer based only on that evidence. This makes the output more grounded, reliable, and useful.

In the AI Research Agent, LLMs are used for final answer generation after hybrid retrieval identifies the most relevant chunks from private PDFs. Lightweight local LLMs running through Ollama make this approach practical for privacy-sensitive and resource-constrained environments. Thus, LLMs play the role of intelligent synthesizers, transforming retrieved context into meaningful answers while still depending on the retrieval system for factual grounding.

CHAPTER 3
Technologies Used

This project uses a combination of Python, NLP libraries, machine learning frameworks, retrieval tools, and local LLM support to build a private document question-answering system. The selected technologies work together to support document ingestion, preprocessing, chunking, embedding generation, retrieval, and final answer generation.

3.1 Python Programming Language

Python is the primary programming language used for the development of the AI Research Agent. It is widely preferred in artificial intelligence, machine learning, and natural language processing because of its simplicity, readability, and large ecosystem of libraries. In this project, Python is used to build the command-line application, implement the ingestion and preprocessing pipeline, train the embedding model, perform retrieval and reranking, and integrate answer generation.

3.2 NLTK for Text Preprocessing

The Natural Language Toolkit, or NLTK, is used for preprocessing and normalizing the extracted text. Since raw PDF text may contain noise, punctuation, repeated words, and multiple grammatical forms, preprocessing is necessary before retrieval. In this project, NLTK is used for sentence tokenization, word tokenization, stopword removal, part-of-speech tagging, and lemmatization. These operations help convert raw text into a cleaner and more searchable form.

3.3 TensorFlow / Keras for Embedding Model

TensorFlow and Keras are used to build and train the custom neural embedding model. The project uses a Siamese-style architecture for learning semantic similarity between chunks of text. This makes it possible to generate vector representations that capture meaning instead of only keyword overlap. Keras was selected because it provides a convenient and flexible way to build neural models such as BiLSTM, BiGRU, and CNN-based encoders.

3.4 scikit-learn for TF-IDF and Retrieval

scikit-learn is used to support lexical retrieval through TF-IDF. While the embedding model handles semantic similarity, TF-IDF improves exact term matching and helps the system perform better on technical documents. In this project, scikit-learn contributes to hybrid retrieval by combining lexical and semantic search, which improves the relevance of the retrieved document chunks.

3.5 PyMuPDF for PDF Text Extraction

PyMuPDF is used for text extraction from PDF files. Since the system works on private PDF documents, accurate extraction is a key requirement. PyMuPDF makes it possible to read each PDF, extract page-level

text, and preserve document-related metadata such as page numbers and source file names. This extracted text is then passed to the preprocessing and chunking stages.

3.6 Ollama for Local LLM Integration

Ollama is used to run lightweight local large language models for answer generation. Instead of relying only on cloud-based APIs, the project supports local inference using small models such as phi3:mini. This makes the system more practical in privacy-sensitive environments and also allows the project to demonstrate LLM-powered RAG functionality locally. Ollama is used after retrieval, where it receives the relevant document context and generates a final grounded answer.

3.7 NumPy for Vector Operations

NumPy is used for numerical and vector-based operations in the retrieval pipeline. Since embeddings are stored and compared as numeric arrays, NumPy helps perform efficient vector manipulation and similarity-related processing. It is therefore an important supporting library in the semantic retrieval stage.

3.8 PyYAML for Configuration Management

PyYAML is used to manage project configuration through the config.yaml file. This allows system settings such as chunk size, overlap, model parameters, retrieval weights, and LLM provider options to be stored outside the main code. As a result, the project becomes easier to configure, test, and maintain.

3.9 Jupyter Notebook for Experimentation

Jupyter Notebook is used for experimentation, exploratory analysis, and model prototyping. It provides an interactive environment in which preprocessing steps, chunk behavior, and embedding model designs can be tested before integrating them into the main application. This is helpful during development and analysis.

Chapter - 4 Design of Project

The design of the AI Research Agent is based on a modular Retrieval-Augmented Generation pipeline for private PDF document question answering. The system is divided into separate stages such as document ingestion, preprocessing, chunking, embedding, retrieval, and answer generation. This modular structure makes the project easier to understand, maintain, and extend in the future.

4.1 Hardware Requirements

The project does not require very high-end hardware and can run on a standard laptop or desktop system. A machine with a modern processor, at least 8 GB RAM, and enough storage for PDF datasets, model files, and outputs is sufficient for normal usage. If larger local LLMs are used through Ollama, better RAM and CPU resources can improve response time.

4.2 Software Requirements

The system is implemented in Python and requires software support such as Python 3.x, TensorFlow, Keras, NLTK, scikit-learn, PyMuPDF, NumPy, and PyYAML. Jupyter Notebook is used for experimentation and analysis, while Ollama is used for lightweight local LLM inference. The project is developed and executed using tools such as VS Code, terminal/command prompt, and GitHub for version control.

4.3 Methodology

The methodology of the project follows a sequential RAG pipeline. First, PDF documents are collected and processed. Then, the extracted text is cleaned and divided into chunks. These chunks are converted into vector representations for retrieval. When a user asks a question, the system retrieves the most relevant chunks using hybrid retrieval and then generates a grounded answer using a local LLM or extractive fallback. This method ensures both contextual relevance and practical usability.

4.3.1 PDF Extraction and Text Processing

The first stage of the methodology is extracting text from private PDF documents. For this purpose, PyMuPDF is used to read PDF files page by page. After extraction, the text is passed into the NLP preprocessing pipeline, where it is normalized through tokenization, stopword removal, POS tagging, and lemmatization. This improves the quality of retrieval by reducing textual noise and inconsistency.

4.3.2 Chunking and Vector Representation

After preprocessing, the extracted text is divided into smaller chunks. Chunking is necessary because long documents are difficult to retrieve and process as a whole. The system uses sentence-aware chunking with overlap so that meaning is preserved across chunk boundaries. These chunks are then converted into vector representations using a trained embedding model, allowing the system to perform semantic search.

4.3.3 Hybrid Retrieval and Answer Generation

When the user submits a query, the system first preprocesses the question and then retrieves relevant chunks using both dense retrieval and lexical retrieval. Dense retrieval uses learned embeddings, while lexical retrieval uses TF-IDF. The results are reranked and passed to the answer-generation stage. A local LLM

running through Ollama generates a final answer based only on the retrieved context, making the response more natural and grounded.

4.4 Implementation

The implementation of the project is organized into separate modules for ingestion, preprocessing, embedding, retrieval, generation, and evaluation. Each part of the system is designed to perform a specific task, which improves clarity and modularity. This implementation style also makes the system easier to update with better retrievers, stronger local models, or user interfaces in the future.

4.4.1 Data Ingestion and Preprocessing Pipeline

The ingestion pipeline begins by scanning the PDF directory and extracting text from each file. The extracted content is then preprocessed using NLTK and stored as structured chunk data. Both raw text and processed text are preserved so that the system can learn from normalized input while still displaying readable answers. This pipeline forms the base of the searchable knowledge store.

4.4.2 Embedding Model Training

The embedding model is trained using a Siamese architecture built in Keras. It learns semantic similarity between chunks so that related content can be retrieved more effectively. During training, chunk pairs are created, encoded into sequences, and used to optimize the model. Once trained, the encoder is saved and used for generating embeddings for retrieval.

4.4.3 Retrieval and Reranking System

The retrieval system combines semantic vector search with lexical TF-IDF search. Dense retrieval helps capture contextual similarity, while lexical retrieval improves exact technical term matching. After retrieval, a reranking step is applied to improve the final ordering of chunks. This makes the system more effective for both short test documents and large technical manuals.

4.4.4 Ollama Integration and Extractive Fallback

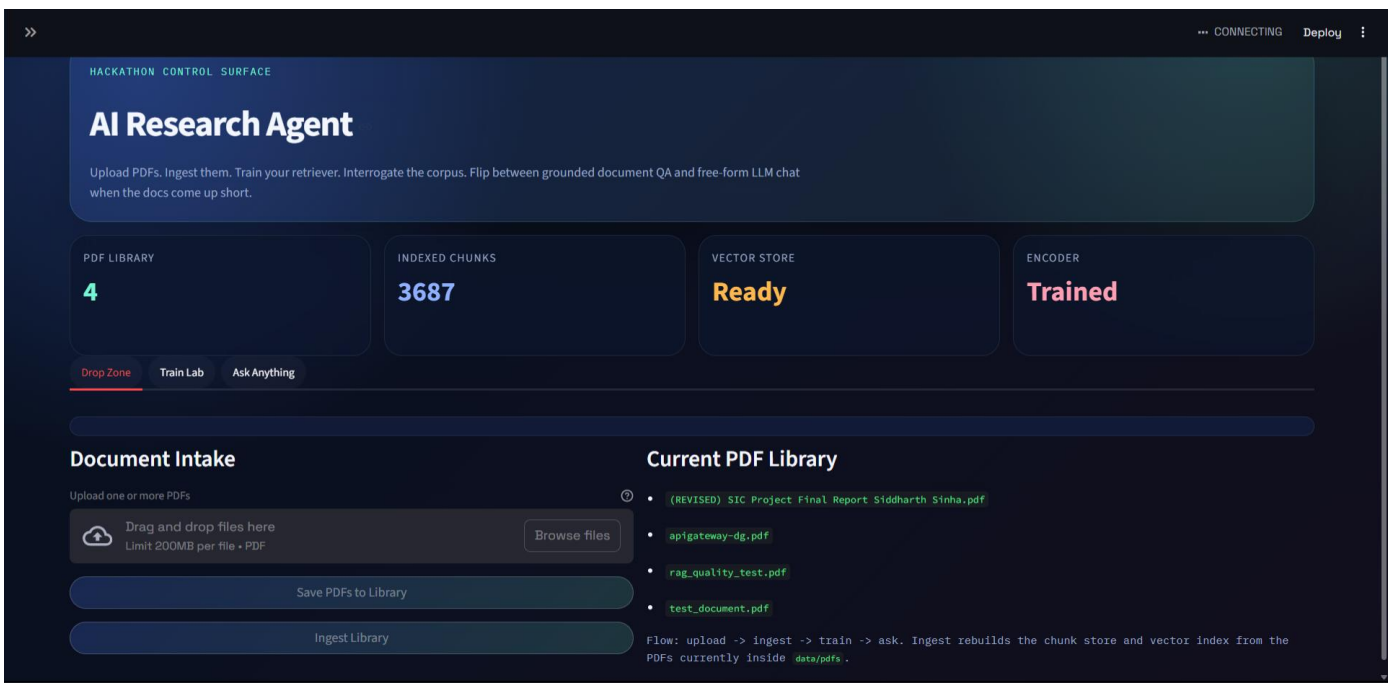
For final answer generation, the project integrates Ollama to run a lightweight local LLM. The retrieved chunks are compressed into a context window and then passed to the model for answer generation. If local generation is unavailable, the system automatically switches to extractive answering, where the best snippets are selected directly from the retrieved chunks. This makes the system both flexible and reliable.

Chapter - 5 Results

The AI Research Agent was successfully developed and tested as an LLM-powered RAG system for private PDF question answering. The final system was able to ingest PDF files, preprocess and chunk the text, train a semantic embedding model, retrieve relevant content using hybrid retrieval, and generate grounded answers using a lightweight local LLM through Ollama. The results show that the project works as a complete end-to-end pipeline.

5.1 System Interface and Setup

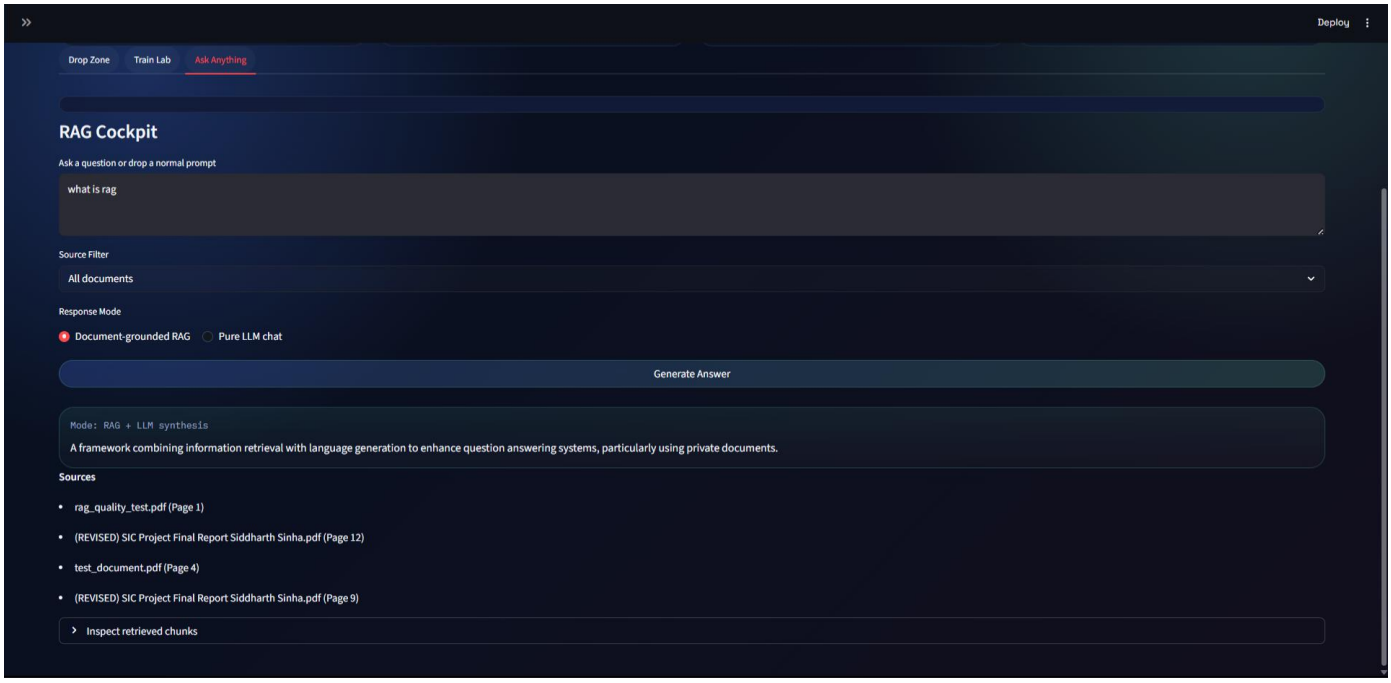
The system was initially implemented as a command-line application, where users can run commands for ingestion, training, querying, and evaluation. This setup made it easy to test the project step by step and verify each module independently. The local LLM integration through Ollama also worked successfully, allowing the system to generate answers without relying entirely on cloud-based APIs. But now has a Combined frontend which allows us to upload & ingest PDFs, train the model, and ask questions.



5.2 Query Processing and Answer Generation Results

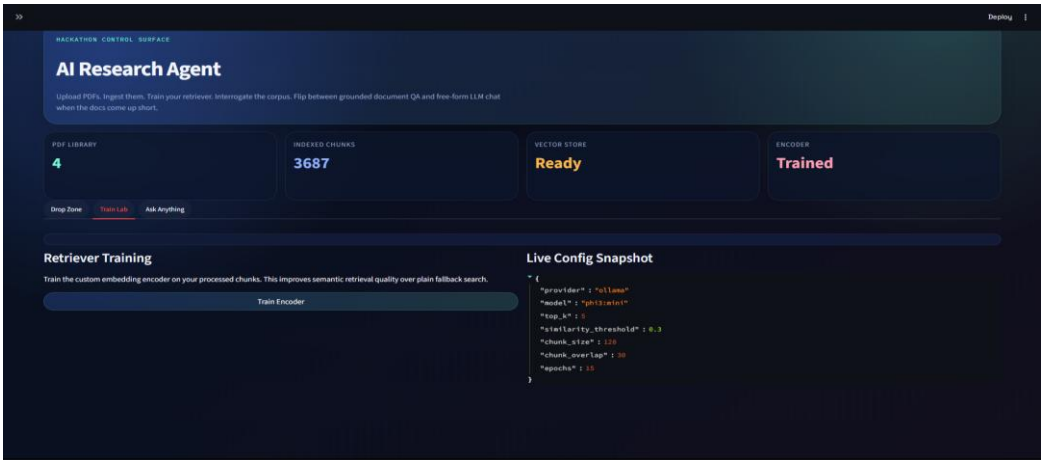
During testing, the system was able to answer questions from uploaded PDF collections in a readable and grounded form. Questions related to smaller and focused PDFs produced more accurate and direct answers, while larger technical PDFs also produced useful results after hybrid retrieval and reranking

were introduced. The generated answers were more natural than extractive-only responses, especially after Ollama integration.



5.3 Retrieval and Model Performance Observations

The retrieval performance improved significantly after combining dense semantic search with lexical TF-IDF search. This hybrid approach made the system better at handling both exact technical keywords and semantically similar queries. The embedding model also improved retrieval quality compared to fallback-only methods. However, it was observed that answer quality still depends strongly on chunk quality, retrieval accuracy, and the complexity of the source PDF documents.



Chapter - 6 Conclusion and Future Scope

The AI Research Agent was successfully developed as an LLM-powered Retrieval-Augmented Generation system for private PDF question answering. The project demonstrated how PDF extraction, NLP preprocessing, chunking, semantic embeddings, hybrid retrieval, and local LLM generation can be combined into one complete framework. The final system was able to process private documents, retrieve relevant context, and generate grounded answers in a practical and efficient manner.

The project also showed the importance of combining classical NLP with modern AI techniques. Preprocessing helped normalize noisy document text, hybrid retrieval improved relevance, and lightweight local LLM generation made the system more useful and interactive. Overall, the system achieved its objective of making private document search and understanding easier through intelligent automation.

In future, the project can be improved by adding a frontend interface, drag-and-drop PDF upload, source highlighting, better reranking models, support for more document formats, and stronger local LLMs. Additional improvements such as conversation history, document filtering, citation display, and feedback-based retrieval tuning can further enhance usability and performance. These extensions would make the framework more scalable, user-friendly, and suitable for real-world deployment.

References

1. Bird, Steven, Edward Loper, and Ewan Klein. *Natural Language Processing with Python*. O'Reilly Media, 2009.
2. Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems*, 2020.
3. Karpukhin, Vladimir, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. "Dense Passage Retrieval for Open-Domain Question Answering." *Proceedings of EMNLP*, 2020.
4. Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." *Proceedings of NAACL-HLT*, 2019.
5. TensorFlow Documentation. *TensorFlow and Keras User Guide*. Available at: https://www.tensorflow.org/
6. NLTK Documentation. *Natural Language Toolkit Documentation*. Available at: https://www.nltk.org/
7. scikit-learn Documentation. *Machine Learning in Python*. Available at: https://scikit-learn.org/
8. PyMuPDF Documentation. *PyMuPDF User Guide*. Available at: https://pymupdf.readthedocs.io/
9. Ollama Documentation. *Ollama Local LLM Platform Documentation*. Available at: https://ollama.com/
10. Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
11. Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. "Attention Is All You Need." *Advances in Neural Information Processing Systems*, 2017.
12. Jurafsky, Daniel, and James H. Martin. *Speech and Language Processing*. 3rd Edition Draft, Stanford University, 2023.